How to: Create GenericPrincipal and GenericIdentity Objects

09/15/2021

> **Note**
>
> This article applies to Windows.
>
> For information about ASP.NET Core, see Overview of ASP.NET Core Security.

You can use the GenericIdentity class in conjunction with the GenericPrincipal class to create an authorization scheme that exists independent of a Windows domain.

To create a GenericPrincipal object

1. Create a new instance of the identity class and initialize it with the name you want it to hold. The following code creates a new **GenericIdentity** object and initializes it with the name `MyUser`.

   C#

   ```
   GenericIdentity myIdentity = new GenericIdentity("MyUser");
   ```

2. Create a new instance of the **GenericPrincipal** class and initialize it with the previously created **GenericIdentity** object and an array of strings that represent the roles that you want associated with this principal. The following code example specifies an array of strings that represent an administrator role and a user role. The **GenericPrincipal** is then initialized with the previous **GenericIdentity** and the string array.

   C#

   ```
   String[] myStringArray = {"Manager", "Teller"};
   GenericPrincipal myPrincipal = new GenericPrincipal(myIdentity, myStringArray);
   ```

3. Use the following code to attach the principal to the current thread. This is valuable in situations where the principal must be validated several times, it must be validated by other code running in your application, or it must be validated by a PrincipalPermission object. You

can still perform role-based validation on the principal object without attaching it to the thread. For more information, see Replacing a Principal Object.

C#

```
Thread.CurrentPrincipal = myPrincipal;
```

Example

The following code example demonstrates how to create an instance of a **GenericPrincipal** and a **GenericIdentity**. This code displays the values of these objects to the console.

C#

```
using System;
using System.Security.Principal;
using System.Threading;

public class Class1
{
    public static int Main(string[] args)
    {
    // Create generic identity.
    GenericIdentity myIdentity = new GenericIdentity("MyIdentity");

    // Create generic principal.
    String[] myStringArray = {"Manager", "Teller"};
    GenericPrincipal myPrincipal =
        new GenericPrincipal(myIdentity, myStringArray);

    // Attach the principal to the current thread.
    // This is not required unless repeated validation must occur,
    // other code in your application must validate, or the
    // PrincipalPermission object is used.
    Thread.CurrentPrincipal = myPrincipal;

    // Print values to the console.
    String name =  myPrincipal.Identity.Name;
    bool auth =  myPrincipal.Identity.IsAuthenticated;
    bool isInRole =  myPrincipal.IsInRole("Manager");

    Console.WriteLine("The name is: {0}", name);
    Console.WriteLine("The isAuthenticated is: {0}", auth);
    Console.WriteLine("Is this a Manager? {0}", isInRole);

    return 0;
```

```
    }
}
```

When executed, the application displays output similar to the following.

Console

```
The Name is: MyIdentity
The IsAuthenticated is: True
Is this a Manager? True
```

See also

- GenericIdentity
- GenericPrincipal
- PrincipalPermission
- Replacing a Principal Object
- Principal and Identity Objects